

Real-Time ML/DL Inference

Day 8 made the consumer agentic with sampled Bedrock enrichment. Today YOUR model joins via SageMaker Serverless. 1 endpoint (the only allowed), no GPUs. Cold-start as a knob. The MODEL CONTEXT block in .windsurfrules v6 enters; 8th HITL dim Confidence Handling.

Day 8 → Day 9

Yesterday Cascade authored the producer + consumer. Today YOUR trained model joins behind a SageMaker Runtime endpoint that scales to zero — same Step Functions orchestration; new payload type.

YESTERDAY

Consumer Lambda with sampled Bedrock enrichment

↓ forks to ↓

TODAY

ASL Parallel: Bedrock branch (existing) + SageMaker branch (new). Both feed Land.

YESTERDAY

BedrockThrottleError in PipelineError ladder

↓ joined by ↓

TODAY

EndpointThrottleException + EndpointColdStartTimeout. Same custom-exception → ASL Retry pattern.

YESTERDAY

STATE CONTEXT block (.windsurfrules → v5)

↓ joined by ↓

TODAY

MODEL CONTEXT block (.windsurfrules → v6) (which model, input shape, confidence threshold, fallback path).

YESTERDAY

7-dim HITL meta-checklist

↓ +1 dim ↓

TODAY

8th dim: Confidence Handling. Catches: silent no-threshold, low-conf-as-data, missing fallback.

YESTERDAY

Lambda count: 8 of 10

↓ +1 today ↓

TODAY

+ wrapper Lambda (invoke_endpoint+postprocess) → 10 of 10 (final cap). Added as SECOND Parallel branch in the EXISTING inference-pipeline state machine; consumer code unchanged.

YESTERDAY

stream + ESM + DLQ + status-model idempotency

↓ unchanged ↓

TODAY

Day 9 inference plugs INTO the consumer; consumer's idempotency contract still owns the record.

Why your own model — Bedrock can't predict airline-specific things

THE GAP — BEDROCK ALONE ISN'T ENOUGH

Foundation models summarise, classify with general categories, explain. They do NOT predict airline-specific outcomes you have private data for: chargeback risk per BSP cycle, ADS-B fault probability per tail number, route demand 14d out per O&D pair, crew duty-time exception likelihood. These need a model trained on YOUR data.

THE CHOICE — SERVERLESS WHEN BURSTY

SageMaker Serverless Inference fits when traffic is bursty/unpredictable, you can tolerate ~1-6s cold-start on first request after idle, you don't want to manage instances, and memory-based pricing makes economic sense vs always-on. NOT for sustained high QPS (use real-time endpoints with provisioned concurrency). Sandbox: 1 endpoint allowed, no GPUs.

By the end of Day 9 you can...

DEPLOY

Stand up a SageMaker Serverless endpoint from a trainer-provided model artefact. Configure memory + max concurrency. Smoke-test.

AUTHOR

Write the MODEL CONTEXT block (.windsurfrules → v6) (which model, input shape, confidence threshold, fallback). Compare to baseline; HITL-score it on 8 dims.

WIRE

Add the endpoint as a Step Functions Task using AWS SDK integration OR a Lambda wrapper (lab uses Lambda wrapper for postprocessing flexibility).

TOLERATE

Handle cold-start + throttling with custom exceptions → ASL Retry/Catch; route low-confidence predictions via a Choice state.

FORK

Refactor consumer's enrich step into ASL Parallel: Bedrock branch + your-model branch → merged result.

MEASURE

Inspect ModelLatency / OverheadLatency / ModelSetupTime via CloudWatch + smoke-test timings (cold-start indicators).

Four parts. Concepts → Design → Labs → Wrap.

01

Concept briefing

Six concepts. Why your model. Serverless endpoint mechanics. Wiring to Step Functions. Confidence as HITL dim. Cold-start + throttle exceptions. The MODEL CONTEXT block in .windsurfrules v6.

TIME

~30 min

02

Guided design

Three guided slides. Endpoint deployment from artefact. Wrapper Lambda contract. ASL Parallel + Choice for low-confidence routing. Cold-start mitigation strategies.

TIME

~35 min

03

Three labs

Lab 1 — Cascade-author CLI to deploy endpoint via 12-layer prompt; smoke-test (45 min). Lab 2 — wrapper Lambda + ASL Parallel fork integration (45 min). Lab 3 — confidence-routing demo: high → enriched/, low → review queue (45 min).

TIME

~135 min

04

Review & wrap

Five Day-9 mistakes. Cleanup that DELETES the endpoint (sandbox cap is 1; D10 may demo too). Day-9 in numbers. Bridge to Day 10 — Observability + the OBSERVABILITY CONTEXT block in .windsurfrules v7.

TIME

~25 min

01

PART 1 · ~30 MINUTES

Concept briefing

Six concepts. Why YOUR model is needed (Bedrock can't predict airline-specific outcomes). Serverless endpoint mechanics + cold-start as a knob. Wiring to Step Functions. Confidence as HITL dim. Custom exceptions for throttle + cold-start. The MODEL CONTEXT block in .windsurfrules v6.

Bedrock summarises and classifies. Your model PREDICTS specific airline outcomes from your data.

Foundation models excel at general categorisation but can't predict things that depend on YOUR private data — chargeback risk per BSP cycle, ADS-B fault for a specific tail number, route demand 14 days out for a specific O&D. Train your own model; deploy it behind a serverless endpoint.

Bedrock vs your-model decision matrix

USE BEDROCK WHEN:

- Free-text summarisation / classification with general categories
- Cancellation reason categorisation (weather/operational/passenger)
- Sentiment / urgency tagging on customer messages
- Metadata extraction from unstructured text

USE YOUR-MODEL WHEN:

- Numeric prediction from YOUR private data
- Examples: chargeback_risk_score per (booking_id, BSP cycle)
 - ADS-B fault probability for a specific tail number
 - Route demand 14d ahead per O&D pair
 - Crew duty-time exception likelihood
- Trained on accelya-historical data (not in any FM training set)
- Output shape: {prediction, confidence, model_version}

DAY 9 DEMO MODEL (trainer-provided):

cancellation_chargeback_risk

Input: {booking_id, fare_amount, refund_lag_hours, prior_cancels}

Output: {risk_score: 0..1, confidence: 0..1, model_version: 'v1.2.3'}

Threshold: confidence $\geq$ 0.7 → use prediction; $<$ 0.7 → route to review

01**Bedrock = general**

Free-text, classification, summarisation.

02**Your-model = specific**

Numeric prediction from private data.

03**Demo: chargeback risk**

Per (booking_id, BSP cycle). Trainer-provided artefact.

04**Output: prediction + conf + version**

Confidence is mandatory; threshold drives routing.

05**Coexist with Bedrock**

ASL Parallel: both branches enrich the same record; downstream merges.

Cold-start ~1-6s on first request after idle. Memory + max concurrency are the two knobs.

SageMaker Serverless Inference scales from 0 to N. Cold-start happens when the endpoint hasn't received traffic recently and AWS recycles the container. You pay only for invocation time + memory.

Endpoint config (lab — sandbox-bound; exact JSON shape required)

```
# EXACT ProductionVariants JSON for create-endpoint-config:
{ "ModelName":      "accelya-cancel-risk-model-<learner>",
  "VariantName":     "AllTraffic",
  "InitialVariantWeight": 1.0,
  "ServerlessConfig": {
    "MemorySizeInMB": 1024,    # valid: 1024 / 2048 / 3072 / 4096 / 5120 / 6144
    "MaxConcurrency": 5        # valid: 1-200
  }
}
# NOTE: For Serverless, do NOT set InitialInstanceCount or InstanceType.
#       Setting them returns ValidationException.

# COLD-START math:
First request after idle: 1-6s ModelSetupTime + ModelLatency
Warm requests:           ModelLatency only (typically 50-300ms)
Idle threshold:          ~5-15 min (AWS-managed)

# CAP REALITY (sandbox):
- 1 endpoint allowed per learner
- No GPU instance types (CPU only on serverless)
- Max 6144 MB memory; max 200 max-concurrency
- Endpoint MUST be deleted at end of session (cumulative endpoint hours billed)
```

01

InstanceCount=0

Serverless. Scales to zero when idle.

02

MemorySizeInMB

1024-6144 valid. Lab uses 1024.

03

MaxConcurrency 5

Sandbox-safe; service cap higher.

04

Cold-start ModelSetupTime

1-6s on first req after idle. CW metric tracks.

05

DELETE at session end

Sandbox cap is 1; cumulative hours billed.

Two ways: AWS SDK Task (direct) or Lambda wrapper. Lab uses wrapper for postprocessing. The Day-8 consumer-to-SFN handoff stays unchanged — D9 only edits the state-machine ASL.

Day 8 already established the THIN consumer + claim → StartExecution → ACK pattern + the inference-pipeline state machine with one Task (BedrockEnrich → Persist). Today we EDIT the ASL to fork BedrockEnrich into ASL Parallel: Bedrock branch (existing) + your-model branch (new wrapper Lambda). Consumer code unchanged. ASL choice — direct SDK Task vs Lambda wrapper — applies only to the new SageMaker branch.

Day 8 handoff contract is the foundation — D9 only adds the second branch

```
## DAY 8 ALREADY ESTABLISHED THIS HANDOFF (kept here for reference):
## THIN consumer:
## 1. Validate envelope shape + idempotency-claim (status-model)
## 2. StartExecution on inference-pipeline state machine (async):
##    DETERMINISTIC execution name: f'enrich-{event_id}' (Day 8 lock)
##    Step Functions Standard idempotent for same name+input WHILE running.
##    ExecutionAlreadyExists treated as accepted (idempotent skip).
## 3. ACK the stream record by marking stream-PK COMPLETED.
##    stream COMPLETED means 'orchestration accepted', NOT 'inference completed'.
##    State machine outputs / DLQ track inference state independently.
##
## TODAY'S CHANGE — Day 8's BedrockEnrich Task forks into ASL Parallel:
## BEFORE (D8): StartAt: BedrockEnrich → Persist
## AFTER (D9): StartAt: Parallel( Bedrock branch + your-model branch ) → Merge → Persist
## The consumer never sees the change — same StartExecution, same exec name, same ACK.

## PATTERN A — AWS SDK direct (no extra Lambda)
{
  "Type": "Task",
  "Resource": "arn:aws:states:::aws-sdk:sagemakerruntime:invokeEndpoint",
  "Parameters": {
    "EndpointName": "accelya-cancel-risk-<learner>",
    "Body.$": "States.JsonToString($.predict_input)",
    "ContentType": "application/json",
    "Accept": "application/json"
  },
  "OutputPath": "$.Body",
  "Retry": [
    {"ErrorEquals": ["SageMaker.ModelError"],
     "IntervalSeconds": 2, "MaxAttempts": 1, "BackoffRate": 2.0},
    {"ErrorEquals": ["States.ALL"], "MaxAttempts": 2}
  ]
}
```

01

Direct SDK

1 Task; no extra Lambda. Limited postprocessing.

02

Lambda wrapper

Lab choice. +1 Lambda (10 of 10 — final cap). Confidence logic + custom exceptions.

03

OutputPath: \$.Body

SDK Task wraps response; strip explicitly.

04

Retry on SageMaker.ModelError

Plus Lambda.* on wrapper-side.

05

Confidence in postprocess

Wrapper computes if conf >= 0.7; emits use_prediction or route_to_review.

If your model returns a confidence and you ignore it, you're shipping garbage with high precision.

Confidence is a property of the prediction. The model says: 'this is my answer + how sure I am'. The pipeline must decide: act on the prediction, or escalate to a human. The 8th HITL dim catches deck/code that ignores this.

Confidence Handling — the 8th HITL dimension

Score 0-3 on these 5 sub-checks per prediction-using artefact:

1. CONFIDENCE THRESHOLD EXPLICIT?

Code/ASL has a numeric threshold (e.g. 0.7). Not 'TBD' or 'tune later'.

2. LOW-CONFIDENCE PATH NAMED?

Where do <threshold predictions go? Review queue? Quarantine? Default value? 'Just log it' isn't a path.

3. CONFIDENCE PERSISTED ALONGSIDE PREDICTION?

Downstream readers see {prediction, confidence}, not just prediction. Without confidence, they can't replicate the high/low decision later.

4. THRESHOLD CALIBRATED?

Is 0.7 chosen on held-out labels or pulled from intuition?
Lab: trainer-calibrated. Production: required.

5. CONFIDENCE-DISTRIBUTION ALERT?

Day 10 wires this. If confidence distribution drifts (mean drops 20%), model degraded. Today: name the metric.

Score: 5 sub-checks × 0-3 = max 15. Pass threshold this dim alone: ≥10/15.

01

Threshold explicit

Numeric, not 'TBD'. Lab default 0.7.

02

Low-conf path named

Review queue / quarantine / default value. Not 'log it'.

03

Persist confidence

Downstream readers see both. Required for replay.

04

Calibrated threshold

Trainer-calibrated for lab; held-out labels for prod.

05

Distribution alert

Day 10 wires; today the metric is named.

EndpointColdStartTimeout + EndpointThrottleException. ASL Retry/Catch routes them.

Serverless endpoints have two predictable failure modes: cold-start that exceeds caller timeout, and throttling under burst. Both need to retry; both need exception names so ASL can decide.

Exception ladder addition (Day 4 + Day 8 + Day 9)

```
## src/common/exceptions.py – Day 9 additions
class EndpointColdStartTimeout(PipelineError):
    """Wrapper Lambda timed out waiting for a cold endpoint to respond.
    Transient – ASL Retry with longer interval."""

class EndpointThrottleException(PipelineError):
    """Endpoint MaxConcurrency exceeded. Transient – backoff."""

## NO LowConfidencePrediction exception – that would conflate normal model
## output with a failure. Wrapper instead returns:
## {prediction, confidence, low_confidence: true|false}
## ASL routes low_confidence via Choice (HasLowConfidence), NOT via Catch.

## Wrapper handler error mapping (sketch)
try:
    response = sm_runtime.invoke_endpoint(...)
except ClientError as e:
    code = e.response['Error']['Code']
    if code == 'ModelError':
        raise PipelineError('model returned error') # generic catch
    if code == 'ServiceUnavailable':
        raise EndpointThrottleException('endpoint throttled')
    if code == 'TimeoutError' or e.response['ResponseMetadata']['HTTPStatusCode']==504:
        raise EndpointColdStartTimeout('cold-start exceeded timeout')
    raise

## ASL routing
EndpointColdStartTimeout → Retry interval=10s, max=2 (cold-start needs longer)
EndpointThrottleException → Retry interval=2s backoff=2.0, max=3
```

01

EndpointColdStartTimeout

Cold-start beyond caller timeout. Retry interval 10s.

02

EndpointThrottleException

MaxConcurrency exceeded. Backoff 2.0x.

03

Low confidence is data, not exception

Wrapper returns low_confidence:true|false flag. ASL Choice (HasLowConfidence) routes — NOT Catch.

04

ASL Retry per exception

Cold-start needs longer interval than throttle.

05

Confidence is HITL routing

Pattern reuses Day 4 quarantine model.

Added to .windsurfrules v5 (becomes v6).

Cascade now reads MODEL CONTEXT before scaffolding any inference wrapper. Tells it which endpoint, input shape, confidence threshold, fallback path.

MODEL CONTEXT layer template

```
## MODEL CONTEXT
Model name:      accelya-cancel-risk
Model version:    v1.2.3 (trainer-provided artefact in s3://.../models/)
Endpoint:         accelya-cancel-risk-<learner> (SageMaker Serverless)
Region:           ap-south-1
MaxConcurrency:   5 (sandbox-safe)
MemorySizeInMB:   1024

Input shape (POST body):
{ booking_id: str, fare_amount: float, refund_lag_hours: float,
  prior_cancels: int }

Output shape:
{ risk_score: 0..1, confidence: 0..1, model_version: str }

Confidence threshold: 0.7 (trainer-calibrated)
Below-threshold action: Wrapper returns low_confidence:true (NORMAL output, NOT exception).
                        ASL Choice 'HasLowConfidence' routes → ReviewQueue + enriched/ (with
flag).
Fallback (endpoint down): set risk_score=null, fallback_used=true,
                        downstream uses Bedrock-derived risk if available

Cold-start expectations:
  ModelSetupTime: 1-6s on first request after idle (~5-15min)
  Wrapper timeout: 30s (= 5× expected cold-start)
  Retry strategy: ASL retry interval 10s × 2 (cold-start needs longer)

Custom exceptions (additions to PipelineError ladder):
  EndpointColdStartTimeout, EndpointThrottleException (NO LowConfidencePrediction – see above)
```

01

Layer slot

Between STATE CONTEXT and CATALOG CONTEXT.

02

Endpoint name + version

Cascade hardcodes against accelya-cancel-risk-<learner> v1.2.3.

03

Confidence threshold 0.7

Numeric, calibrated. Below → low_confidence:true flag (data, not exception).

04

Fallback explicit

Endpoint down → null + Bedrock fallback used.

05

Forbidden in layer

Multiple endpoints, GPUs, ignoring confidence.

Inference vocabulary set

Bedrock vs your-model. Serverless mechanics + cold-start. Wiring patterns. Confidence as 8th HITL dim. Custom exceptions for throttle/cold-start. The MODEL CONTEXT block in .windsurfrules v6.

VOCABULARY

MaxConcurrency, ModelSetupTime, ModelLatency, EndpointColdStartTimeout, EndpointThrottleException, low_confidence flag, HasLowConfidence Choice, ProvisionedConcurrency.

WHAT'S NEW

MODEL CONTEXT block in .windsurfrules v6. 8th HITL dim (Confidence Handling). 3 new custom exceptions. Wrapper Lambda pattern.

WHAT'S NEXT

Three guided design slides: endpoint deployment, wrapper Lambda contract, ASL Parallel + Choice for low-confidence routing.

02

PART 2 · ~35 MINUTES

Guided design

Three guided design slides. Endpoint deployment from trainer-provided artefact. Wrapper Lambda contract (input/output + confidence handling). ASL Parallel fork (Bedrock + your-model branches) + Choice for low-confidence routing.

Three commands: create-model → create-endpoint-config → create-endpoint. Wait for InService.

STEP 1 — create-model

Points at trainer-provided artefact tarball.

```
MODEL_ARTEFACT_S3=s3://accelya-airline-data-ap-south-1/models/cancel-risk/v1.2.3/model.tar.gz
INFERENCE_IMAGE=246618743249.dkr.ecr.ap-south-1.amazonaws.com/sagemaker-scikit-learn:1.2-1-cpu-py3

aws sagemaker create-model \
  --model-name accelya-cancel-risk-model-$LEARNER \
  --primary-container
Image=$INFERENCE_IMAGE,ModelDataUrl=$MODEL_ARTEFACT_S3 \
  --execution-role-arn $SAGEMAKER_ROLE_ARN \
  --region ap-south-1
```

STEP 2 — endpoint-config (serverless)

MemorySizeInMB + MaxConcurrency. No GPU; CPU image only (sandbox).

```
aws sagemaker create-endpoint-config \
  --endpoint-config-name accelya-cancel-risk-config-$LEARNER \
  --production-variants '[{
    "VariantName":"AllTraffic",
    "ModelName":"accelya-cancel-risk-model-$LEARNER",
    "InitialVariantWeight":1.0,
    "ServerlessConfig":{"
      "MemorySizeInMB":1024,
      "MaxConcurrency":5
    }}]' \
  --region ap-south-1
# Note: VariantName='AllTraffic' (not 'primary') – convention for single-variant serverless.
# MemorySizeInMB valid:
1024/2048/3072/4096/5120/6144. MaxConcurrency:
1-200.
```

STEP 3 — create-endpoint + wait

Status starts Creating; wait for InService (~3-5 min).

```
aws sagemaker create-endpoint \
  --endpoint-name accelya-cancel-risk-$LEARNER \
  --endpoint-config-name accelya-cancel-risk-config-$LEARNER \
  --region ap-south-1

# Poll for InService:
while [ "$(aws sagemaker describe-endpoint \
  --endpoint-name accelya-cancel-risk-$LEARNER --region ap-south-1 \
  --query 'EndpointStatus' --output text)" !=
  'InService' ]; do
  echo waiting...; sleep 30
done
```

Single-record wrapper. Step Functions Task invokes once per consumer-batched record.

INPUT / OUTPUT SHAPE

Wrapper accepts the consumer's per-record predict_input; returns enriched payload.

```
## Input (from ASL Parallel branch – per record)
{ booking_id: 'BK-2026-...', fare_amount: 2845.0,
  refund_lag_hours: 2.5, prior_cancels: 0 }

## Output
{ risk_score: 0.42, confidence: 0.81,
  model_version: 'v1.2.3',
  low_confidence: false, model_setup_ms: 0,
  model_latency_ms: 124 }
# low_confidence is a NORMAL OUTPUT FLAG, not an exception.

## On low confidence (conf < threshold): just emit low_confidence=true.
ASL Choice (HasLowConfidence) downstream routes to review queue + enriched/.

## On endpoint down / cold-start timeout:
return {risk_score: null, fallback_used: true, ...}
# Downstream uses Bedrock-derived risk if available
```

CORE WRAPPER LOGIC

boto3 SageMakerRuntime + EXPLICIT socket timeouts + body parsing checklist + custom exception mapping.

```
import boto3, json, time, os
from botocore.config import Config
from botocore.exceptions import ClientError, ReadTimeoutError, ConnectTimeoutError
from common.exceptions import EndpointColdStartTimeout, EndpointThrottleException, PipelineError
# NOTE: NO LowConfidencePrediction. Low confidence is a normal model output, not an exception.

# EXPLICIT boto3 socket timeouts – defaults (60s/60s) exceed Lambda timeout.
# connect_timeout=5s: TCP handshake to SM Runtime endpoint should be <1s.
# read_timeout=35s: must be < Lambda timeout (45s) so we get ReadTimeoutError
# instead of Lambda runtime kill (which gives no useful exception).
sm = boto3.client('sagemaker-runtime', region_name='ap-south-1',

config=Config(connect_timeout=5, read_timeout=35,

retries={'max_attempts': 0})) # ASL owns retry
ENDPOINT = os.environ['ENDPOINT_NAME']
THRESHOLD =

float(os.environ.get('CONFIDENCE_THRESHOLD', '0.7'))

def handler(event, context):
```

WRAPPER LAMBDA CONFIG

Memory + timeout sized for cold-start tolerance — EXPLICITLY larger than boto3 read_timeout.

```
Function:    accelya-predict-cancel-risk-$LEARNER
Runtime:     python3.12
Memory:      512 MB      # wrapper is light; SageMaker is the heavy work
Timeout:     45s         # MUST be > boto3 read_timeout (35s) + parse buffer.
                                # Cold-start ceiling 6s
                                # +10s ASL invoke overhead margin. Avoid 60s (= 2x p99 # cold-start budget; would mask runaway endpoint).
Environment:
  ENDPOINT_NAME:          accelya-cancel-risk-$LEARNER
  CONFIDENCE_THRESHOLD:   0.7

# Why these three numbers in lockstep:
#   Lambda 45s > boto3 read_timeout 35s > ASL retry interval 10s
#   So a slow cold-start surfaces as ReadTimeoutError → EndpointColdStartTimeout
#   → ASL Retry, NOT as Lambda Runtime.HandlerNotFound (silent kill).
```


Refactor consumer's enrich step to ASL Parallel; route low-confidence via Choice.

ASL PARALLEL — TWO BRANCHES

Bedrock branch + SageMaker branch run concurrently.
Both feed Merge.

```
{
  "Type": "Parallel",
  "Branches": [
    {
      "StartAt": "BedrockEnrich",
      "States": {
        "BedrockEnrich": {
          "Type": "Task",
          "Resource": "arn:...:lambda:invoke",
          "Parameters": {
            "FunctionName": "accelya-bedrock-enrich-
<learner>",
            "Payload.$": "$",
            "OutputPath": "$.Payload", "End": true
          }
        }
      },
    },
    {
      "StartAt": "PredictCancelRisk",
      "States": {
        "PredictCancelRisk": {
          "Type": "Task",
          "Resource": "arn:...:lambda:invoke",
          "Parameters": {
            "FunctionName": "accelya-predict-cancel-risk-
<learner>",
            "Payload.$": "$",
            "OutputPath": "$.Payload",
            "Retry": {
              "IntervalSeconds": 10,
```

MERGE STATE

Combines both branch outputs into a single enriched record.

```
"MergeEnrichment": {
  "Type": "Task",
  "Resource": "arn:...:lambda:invoke",
  "Parameters": {
    "FunctionName": "accelya-merge-enrichment-
<learner>",
    "Payload": {
      "original.$": "$.original",
      "bedrock_result.$": "$.parallel[0]",
      "model_result.$": "$.parallel[1]"
    }
  },
  "OutputPath": "$.Payload",
  "Next": "PersistEnriched"
}

# Lab choice: Day 8 consumer Lambda already
does the persist; in Lab 2 we
# refactor it to be the merge+persist function.
No new Lambda for merge.
```

CHOICE STATE — ROUTE LOW-CONFIDENCE

After merge, Choice gates: high-confidence → enriched/
low-confidence → review queue.

```
"HasLowConfidence": {
  "Type": "Choice",
  "Choices": [
    { "Variable": "$.model_result.decision",
      "StringEquals": "low_confidence_review",
      "Next": "WriteReviewQueue" }
  ],
  "Default": "WriteEnriched"
}

# DURABLE SINK — low-confidence ALWAYS lands
somewhere persistent.
# TWO sinks: SQS for human review + S3 for
audit (always-write enriched, with flag).
"WriteReviewSinks": {
  "Type": "Parallel",
  "Branches": [
    { "StartAt": "WriteSQS", "States": {
      "WriteSQS": { "Type": "Task",
        "Resource": "arn:...:sqs:sendMessage",

        "Parameters": { "QueueUrl": "$MODEL_MODEL_REVIEW_Q
UEUE_URL",

        "MessageBody.$": "States.JsonToString($)",
          "End": true } } },
    { "StartAt": "WriteAuditS3", "States": {
      "WriteAuditS3": { "Type": "Task",
        "Resource": "arn:...:s3:putObject",

        "Parameters": { "Bucket": "...",
```

Endpoint + wrapper + ASL all designed

Endpoint deploy 3-step. Wrapper Lambda contract with confidence-threshold raise. ASL Parallel + Choice for low-confidence routing.

ENDPOINT DEPLOY

create-model → create-endpoint-config → create-endpoint. Wait InService.

WRAPPER LAMBDA

Light (512MB, 30s). Maps 3 custom exceptions. Raises LowConfidencePrediction on conf<0.7.

ASL PARALLEL+CHOICE

Two branches (Bedrock + Model). Catch routes LowConfidence; Choice routes review-queue post-merge.

03

PART 3 · ~135 MINUTES

Three labs

Lab 1 (45min) — deploy SageMaker Serverless endpoint via Cascade-authored 3-command script. Lab 2 (45min) — wrapper Lambda + ASL Parallel fork. Lab 3 (45min) — confidence routing demo: high → enriched/, low → review queue.

Deploy SageMaker Serverless endpoint

BRIEF

Pair-work. Use prompts/v12_model.txt (12-layer with MODEL CONTEXT). Cascade-author scripts/deploy_m9_endpoint.sh that runs the 3-step deploy. Wait for InService. Smoke-test with sample input. Capture endpoint ARN + smoke-test JSON. Note ModelSetupTime cold-start.

TIME

45 min

LEVEL

Working+

DELIVERABLE

prompts/v12_model.txt (12-layer with MODEL CONTEXT)
scripts/deploy_m9_endpoint.sh
/reviews/m9-endpoint-smoke-test.md (endpoint ARN + smoke response + cold-start ModelSetupTime metric)

Six steps. Branch → v12 prompt → Cascade-author deploy script → run + wait → smoke test → MR.

1

Branch + 12-layer prompt

```
export LEARNER=jai
git checkout main && git pull --ff-only
git checkout -b module-9-$LEARNER
cd prompts/v12_model.txt
```

EXPECTED

On module-9-<learner>. v12_model.txt has 12 layers; MODEL CONTEXT visible.

2

Cascade-authors deploy + reset scripts

```
# In Windsurf chat: attach prompts/v12_model.txt
# TASK 1: scripts/reset_module9_endpoint.sh - IDEMPOTENT cleanup.
# delete_endpoint → poll until EndpointStatus is gone
# delete_endpoint-config → delete-model
```

EXPECTED

scripts/deploy_m9_endpoint.sh + scripts/reset_module9_endpoint.sh present.

3

Run + wait for InService

```
bash scripts/deploy_m9_endpoint.sh $LEARNER
# Outputs ENDPOINT_ARN.
aws sagemaker describe-endpoint --endpoint-name accelva-cancel-risk-$LEARNER \
# { "modelName": <model>, "variantName": AllInpattic,
```

EXPECTED

EndpointStatus InService. Endpoint ARN captured.

4

Smoke-test with sample input (cold-start measurement)

```
PAYLOAD='{ "booking_id": "BK-TEST-001", "fare_amount": 2845.00, "refund_lag_hours": 2.5, "prior_cancels": 0 }'
aws sagemaker-runtime invoke-endpoint \
--endpoint-name accelva-cancel-risk-$LEARNER \
--payload $PAYLOAD
```

EXPECTED

Response includes risk_score + confidence + model_version. ModelSetupTime CW metric > 0 (cold-start observed).

5

Capture cold-start metric

```
aws cloudwatch get-metric-statistics --namespace AWS/SageMaker \
--metric-name ModelSetupTime \
--dimensions Name=EndpointName,Value=accelva-cancel-risk-$LEARNER \
--start-time $(date +%Y-%m-%d %H:%M:%S)
```

EXPECTED

ModelSetupTime non-zero on first call. Latency on second call (warm) much lower.

6

Document + commit

```
# /reviews/m9-endpoint-smoke-test.md captures endpoint ARN + response + ModelSetupTime
git add scripts/ prompts/v12_model.txt reviews/m9-endpoint-smoke-test.md
git commit -m "module-9: endpoint deployed + smoke test (Lab 1)"
git push origin module-9-$LEARNER
```

EXPECTED

MR open; pair partner tagged.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- Endpoint InService with ServerlessConfig (Memory=1024, MaxConcurrency=5)
- Smoke test returns valid JSON with risk_score + confidence + model_version
- ModelSetupTime CW metric shows cold-start on first call
- /reviews/m9-endpoint-smoke-test.md captures endpoint ARN + response + cold-start metric
- MR open; pair partner tagged

⚠ TOP GOTCHAS

Wrong inference image (GPU)

Sandbox blocks GPU. Use CPU sklearn image.

MaxConcurrency too high

Sandbox-safe at 5. Avoid 200 (account quota check).

Forgetting to wait for InService

describe-endpoint immediately returns Creating. Poll until InService.

Wrong --cli-binary-format

Without raw-in-base64-out, AWS CLI base64-encodes the JSON body and SageMaker rejects.

Endpoint left running

Sandbox cap is 1; cumulative endpoint hours billed. Lab 3 cleanup MUST delete.

Wrapper Lambda + ASL Parallel fork integration

BRIEF

Pair-work. Cascade-author src/predict/handler.py (wrapper) per MODEL CONTEXT. Add EndpointColdStartTimeout / EndpointThrottleException / LowConfidencePrediction to common/exceptions.py. Refactor consumer ASL: replace single Bedrock-enrich Task with ASL Parallel (Bedrock branch + your-model branch). Deploy. Send 1 test event end-to-end.

TIME

45 min

LEVEL

Working+

DELIVERABLE

src/predict/handler.py + tests
Updated infra/validator-pipeline.asl.json with
Parallel + Choice
/reviews/HITL_REVIEW-m9-inference.md (8 dims
incl. Confidence Handling)

Success criteria & common gotchas

✓ YOU ARE READY IF...

- Wrapper Lambda Active (count cumulative 10 of 10 — final cap reached)
- infra/validator-pipeline.asl.json updated with Parallel + Choice + Catch routes
- ASL Catch route specifically for LowConfidencePrediction → ReviewQueue Pass state
- End-to-end test execution SUCCEEDED with both Parallel branches visible in history
- HITL review scored $\geq 19/24$ across 8 dims (incl. Confidence Handling 5 sub-checks)
- MR shows 2 commits

⚠ TOP GOTCHAS

Calling SageMaker from consumer

MODEL CONTEXT forbids it. Always wrapper Lambda Task.

Ignoring confidence

8th HITL dim. Pipeline must EITHER use the prediction OR route low-confidence.

Wrapper timeout too short

Cold-start 1-6s × cushion → 30s. 10s is risky.

Catch loops

ReviewQueue is Pass + End. Never re-enter PredictCancelRisk; that loops.

Hardcoding endpoint name

Use env var ENDPOINT_NAME; otherwise can't reuse wrapper post-deploy rename.

Confidence routing demo + endpoint cleanup

BRIEF

Solo. Send 10 events: 7 with profiles that produce confidence > 0.7 (use prediction); 3 with low-confidence profiles (route to review queue). Verify in SQS review queue: 3 messages. Verify enriched/: 7 records with risk_score populated. DELETE the endpoint at end (sandbox cap is 1; cumulative billed).

TIME

45 min

LEVEL

Working

DELIVERABLE

/reviews/m9-confidence-routing-evidence.md (7+3 split + SQS depth + enriched/ count)
Endpoint DELETED

Six steps. Generate 7+3 events → send → trace → verify SQS → verify enriched/ → DELETE endpoint.

1

Generate 10 events (7 high-conf + 3 low-conf)

```
python3 -m scripts.gen_m9_test_batch \
  --high-conf 7 --low-conf 3 \
  --out /tmp/m9-10.json
```

EXPECTED

10 events; profile difference makes 3 produce conf<0.7.

2

Provision review SQS + send via producer

```
aws sqs create-queue --queue-name model-review-queue-$LEARNER --region ap-south-1
MODEL_REVIEW_QUEUE_URL=$(aws sqs get-queue-url --queue-name model-review-queue-$LEARNER \
  --region ap-south-1 --query 'QueueUrl' --output text)
# Set MODEL_REVIEW_QUEUE_URL in state-machine context (re-deploy ASL if needed)
```

EXPECTED

10 records flow through pipeline; 30-60s for ESM + ASL.

3

Verify review queue depth

```
aws sqs get-queue-attributes --queue-url $MODEL_REVIEW_QUEUE_URL \
  --attribute-names ApproximateNumberOfMessages --region ap-south-1
```

EXPECTED

ApproximateNumberOfMessages = 3 (the low-confidence trio).

4

Verify enriched/

```
aws s3 ls s3://accelya-airline-data-ap-south-1/enriched/$LEARNER/ \
  --recursive --region ap-south-1 | tail -10
```

EXPECTED

7 new objects with risk_score + confidence + model_version populated.

5

Document evidence

```
# Capture: 7 enriched / 3 review queue / counts match expected.
git add reviews/m9-confidence-routing-evidence.md
git commit -m "module-9: confidence routing evidence (Lab 3)"
git push
```

EXPECTED

/reviews/m9-confidence-routing-evidence.md committed.

6

DELETE the endpoint (CRITICAL — sandbox cap 1)

```
aws sagemaker delete-endpoint --endpoint-name accelya-cancel-risk-$LEARNER \
  --region ap-south-1
aws sagemaker delete-endpoint-config --endpoint-config-name accelya-cancel-risk-config-$LEARNER \
  --region ap-south-1
aws sagemaker delete-model --model-name accelya-cancel-risk-model-$LEARNER \
  --region ap-south-1
```

EXPECTED

Endpoint deleted. Endpoint-config + model also deleted (cleanup).

Success criteria & common gotchas

✓ YOU ARE READY IF...

- 7 high-conf events landed in enriched/ with populated risk_score + confidence + model_version
- 3 low-conf events in review-queue-<learner> SQS
- /reviews/m9-confidence-routing-evidence.md captures evidence
- Endpoint + endpoint-config + model ALL deleted
- MR shows 3 commits; pair partner tagged

⚠ TOP GOTCHAS

Forgetting endpoint deletion

Sandbox cap is 1; cumulative billed. ALWAYS delete at session end.

Wrong sample profile

If all 10 events score high-conf, low-conf path untested. Trainer-provided generator handles this.

MODEL_REVIEW_QUEUE_URL not set

ASL WriteReviewQueue Task fails on missing env var. Re-deploy ASL with the URL.

SQS visibility leaving messages stuck

Review queue is for human read; default visibility 30s is fine.

Skipping the cleanup verification

ALWAYS run list-endpoints to confirm; otherwise sandbox blocks D11/D12 endpoint creation.

Code shipped. Branches pushed. Time to step back.

04

PART 4 · ~25 MINUTES

Review & wrap

Five Day-9 inference mistakes. Cleanup that DELETES the endpoint (sandbox cap 1). Day-9 in numbers. Bridge to Day 10 — Observability, Security & Cost. OBSERVABILITY CONTEXT block in .windsurfrules v7.

Five Day-9 inference mistakes — and what to do instead

01

Calling SageMaker synchronously from the consumer

MISTAKE Consumer Lambda calls `invoke_endpoint` inline. Couples consumer SLA to model cold-start; ESM rewinds on slow inference; double-process risk. (Day 8 explicitly prevented this with the THIN consumer rule — but the temptation creeps back when Cascade is asked to

DO INSTEAD

Consumer ONLY validates + idempotency-claims + `StartExecution` on inference-pipeline state machine (async) + ACKs stream record. State machine runs Parallel + Merge + Choice. Decoupled.

02

No confidence threshold

MISTAKE Wrapper returns prediction regardless of confidence. Downstream uses noisy predictions silently.

DO INSTEAD

`CONFIDENCE_THRESHOLD` env var; wrapper emits `low_confidence:true|false` flag (NOT exception); ASL Choice (`HasLowConfidence`) routes — review queue + enriched/ for low-conf, enriched/ only for high-conf.

03

Multiple endpoints (sandbox blocks)

MISTAKE Trying to deploy v1.2.3 alongside v1.2.4 for A/B. Sandbox blocks 2nd endpoint.

DO INSTEAD

Lab uses 1 endpoint per learner. Production A/B uses `ProductionVariants` on a single endpoint with traffic split.

04

`invoke_endpoint` without explicit `ContentType`

MISTAKE boto3 default `ContentType` doesn't match the model's expected. Returns 415 Unsupported Media Type.

DO INSTEAD

ALWAYS pass `ContentType='application/json'` AND `Accept='application/json'`.

05

Endpoint left running

MISTAKE Sandbox cap is 1; cumulative endpoint hours billed. Day 11 / Day 12 can't deploy until you delete.

DO INSTEAD

Lab 3 step 6 deletes endpoint + endpoint-config + model. Verify with `list-endpoints` showing 0 results matching `$LEARNER`.

End-of-module cleanup

CLEANUP CHECKLIST

- Branch pushed: module-9-<learner>; MR with 3 commits; pair partner tagged
- ENDPOINT DELETED (CRITICAL — sandbox cap is 1; D10/D11/D12 need the slot)
- endpoint-config + model also deleted (sandbox cleanup discipline)
- Wrapper Lambda Active (count cumulative 10 of 10 — final cap reached; D11 PreTraffic Hook is alarm-based, no extra Lambda)
- ASL state machine retains Parallel + Choice (D10 dashboards inherit)
- review-queue-<learner> SQS preserved (D10/D12 may use)
- Bedrock invocations: ~35 of 50 used (sampled enrichment in Lab 2 + Lab 3); 15 left for D10/D11
- stream + DDB + IAM (D7+D8 inheritance) preserved

```
# Verify endpoint deletion (CRITICAL):
aws sagemaker list-endpoints --region ap-south-1 \
  --query "length(Endpoints[?contains(EndpointName,'$LEARNER')])"
# Must output: 0

aws lambda list-functions --region ap-south-1 \
  --query "length(Functions[?contains(FunctionName,'$LEARNER')])"
# Must output: 10 of 10 (final cap reached)

# Bedrock budget (3 metrics):
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0
for METRIC in Invocations InputTokenCount OutputTokenCount; do
  aws cloudwatch get-metric-statistics --namespace AWS/Bedrock \
    --metric-name $METRIC --dimensions Name=ModelId,Value=$PROFILE \
    --start-time $(date -u -d 'today 00:00' +%FT%T) \
    --end-time $(date -u +%FT%T) --period 3600 --statistics Sum \
    --region ap-south-1
done
```

Trainer note: endpoint MUST be deleted; sandbox cap blocks future deploys otherwise. Lambda count 10 of 10 — final cap reached. D11 PreTraffic Hook uses ALARM-BASED rollback (no extra Lambda). Bedrock ~30/50 — D10/D11 need ~15.

What you actually shipped

1

ENDPOINT DEPLOYED (then deleted)

SageMaker Serverless.
ModelSetupTime cold-start observed.
Cleanup discipline.

10/10

LAMBIDAS (FINAL CAP)

+1 wrapper. Cap reached. D11
PreTraffic Hook uses alarm-based
rollback (no extra Lambda).

1

ASL FORK

Parallel: Bedrock + your-model.
Choice: high-conf → enriched / low-
conf → review queue.

12

PROMPT LAYERS

+ MODEL CONTEXT. 8 HITL dims (+
Confidence Handling).

Day 10 — Observability, Security & Cost

Today the system is too complex to debug by tail-ing CloudWatch. Tomorrow: 3 pillars, EMF, X-Ray across SF + Lambda + Bedrock + SageMaker, IAM tightening, cost as observability. The OBSERVABILITY CONTEXT block in .windsurfrules v7 enters.

DASHBOARDS + ALARMS

0 new Lambdas; instrumentation only

Add powertools Logger + correlation IDs across all 9 Lambdas. Build CW dashboard. SNS alarms on Bedrock throttle, IteratorAge, confidence drift.

13TH PROMPT LAYER

OBSERVABILITY CONTEXT (correlation_id, EMF, trace propagation)

Cascade scaffolds with observability hooks built-in.

SANDBOX BUDGET

0 new Lambdas + ~5 Bedrock invocations (Cascade-led)

Cumulative: still 9 of 10 Lambdas; ~40 of 50 Bedrock used.

Day 10 branch: module-10-<learner>. Inherits .windsurfrules v6 (with MODEL CONTEXT). 9 active Lambdas + ASL state machine + workgroup all become observable.

DAY 9 · WRAPPED

Your model is in the pipeline.

ENDPOINT + WRAPPER

SageMaker Serverless deployed + wrapper Lambda with confidence threshold. ASL Parallel fork live.

8TH HITL DIM

Confidence Handling. 5 sub-checks per artefact. Catches silent-no-threshold + missing fallback.

.WINDSURFRULES v7

MODEL CONTEXT block in .windsurfrules v6. 3 new exceptions. Forbidden patterns named.

Day 10 starts at 09:00. Bring your 9 Lambdas + ASL + workgroup. Observability enters; the OBSERVABILITY CONTEXT block in .windsurfrules v7 joins the .windsurfrules v6 you used today.

Accelya Kale · Agentic Development using AWS · Trainer: Jai Singh